

Software Design:

The GameEnvironment class is our abstract base class, which has 4 derived concrete classes: PhaseOne, PhaseTwo, PhaseThree, and PhaseFour. Each of these “Phase” classes corresponds with a certain stage of our game. The Puzzles class is related to the GameEnvironment class through aggregation, and a 1..*-to-1 relationship. The class itself focuses on creating the puzzles that the player will utilize to complete the game. On the other side of the UML diagram, the HelpMenu class resides. Within this class resides methods that assist the player if they hit a roadblock in the game. It’s associated with the Player class with a one-to-one relationship. Within the Player class contains methods to create a player in terms of their name, position in the game map, and initializing an inventory for them to store their items. This class is also associated with the NPC (with a 1-to-1.* relationship), Items (with a 1-to-1 relationship), and Command (with a 1-to-1 relationship) classes. Below the Player class on the UML diagram is the NPC class, which initializes various NPC’s depending on the current phase of the game the player is in. The NPC class relates to the Items class through composition – and a 1-to-1 relationship – and the GameEnvironment class – through aggregation and a 1..*-to-1 relationship. The Items class is on the top of the UML diagram and handles the items that the NPC gives to the player and regular items that the player can find and pick up. The other class that the Items class relates to is the GameEnvironment with an aggregation relationship and a 1..*-to-1 relation. The Command class controls the movement of the player and is additionally connected with the GameEnvironment class with a 1-to-1 relationship. Lastly, is the Exceptions class in the upper left corner of the UML diagram. This is a class that will contain exceptions to be thrown, allowing the game to properly error-check and prevent itself from crashing.

Design Rationale:

Although our design may undergo some changes when we began to implement the code, the choices that we have made satisfy the goal of our game and help it to run in the way that we want it too. Additionally, our design also satisfies the SOLID and DRY principles. To begin, our design passes the “Single Responsibility Principle”, as every object has a single responsibility (shown through the specifics of each class within our UML diagram) and all the methods within these classes focus on carrying out said responsibility. For example, the Puzzles class sole purpose is to create and distribute puzzles for the player to use. Additionally, all our classes follow the “Open-Closed Principle”, as the actual classes are abstracting what’s common and encapsulating the differences, such as in the classes GameEnvironment, PhaseOne, PhaseTwo, PhaseThree, and PhaseFour. Additionally, it passes the “Dependency Inversion Principle”, as details depend on abstractions, and the

high-level modules should not depend on the low-level modules. This is also clearly shown through the implantation of the GameEnvironment, PhaseOne, PhaseTwo, PhaseThree, and PhaseFour classes. Furthermore, it also satisfies the “Liskov Substitution Principle” as the subtypes must be substitutable for their base types and it involves well-designed inheritance. This is, once again, showcased through our abstract base class GameEnvironment and it’s four derived base classes. The “Interface Segregation Principle” is also satisfied, as no client is forced to depend on methods that don’t use. This is also shown through the implementation of GameEnvironment and its derived classes, as each of the derived classes choose the functionalities that matter to them, such as the printDescription and victoryCondition methods. Lastly, it follows the “Design Testing” principle, as it separates the classes into separate ones. Additionally, as we implement the code, we can focus on expanding more if necessary. In terms of DRY principles, this also is applied in our design, as everything is in a sensible, as nothing is repeated and there is no duplicate code present.